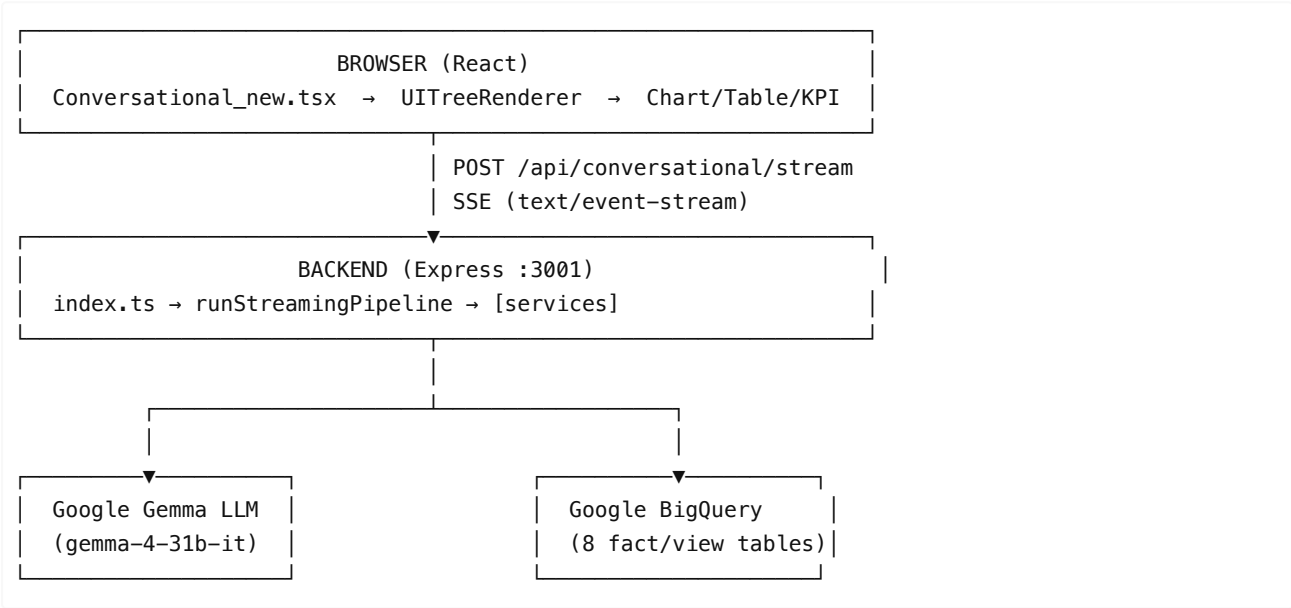


Chat Flow Architecture — conversationUI

Overview

End-to-end flow: user types query → SSE streaming pipeline → LLM generates UI tree → BigQuery hydrates data → React renders components incrementally.

System Layers



1. Frontend Entry & Routing

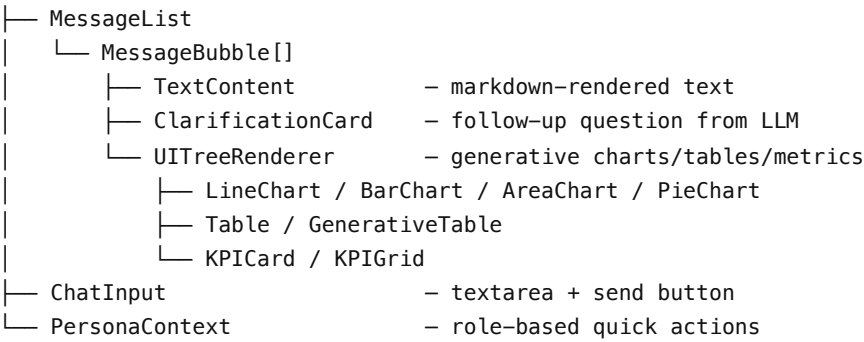
File	Role
src/main.tsx	React DOM root mount
src/app/App.tsx	BrowserRouter — 15+ routes
src/app/pages/Conversational_new.tsx	Primary chat page (2700+ lines)

Key routes:

- `/conversational` — main chat UI
- `/report-flow` — guided 5-step report creation
- `/migration` — dataset migration assistant

2. Chat UI Component Tree

Conversational_new.tsx	
├ ConversationSidebar	— list of saved conversations
└ SuggestedPrompts	— context-aware quick prompts



3. State Management

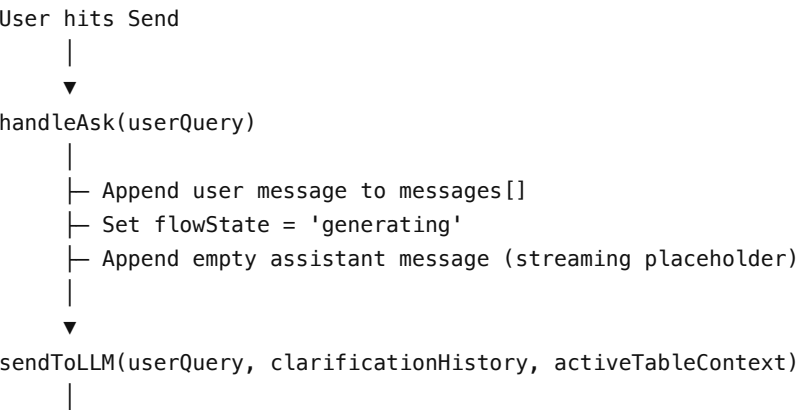
No external store (Redux/Zustand). All state lives in `Conversational_new.tsx` via `useState` .

State Variable	Type	Purpose
messages	Message[]	All messages in active conversation
conversations	Conversation[]	Sidebar conversation list
activeConversationId	string	Currently selected conversation
flowState	enum	idle / clarifying / generating / error
clarificationHistory	ClarificationTurn[]	Multi-turn clarification context
createReportState	object	Guided report creation wizard state
migrationState	object	Dataset migration workflow state
isGenerating	boolean	SSE stream in progress

Context:

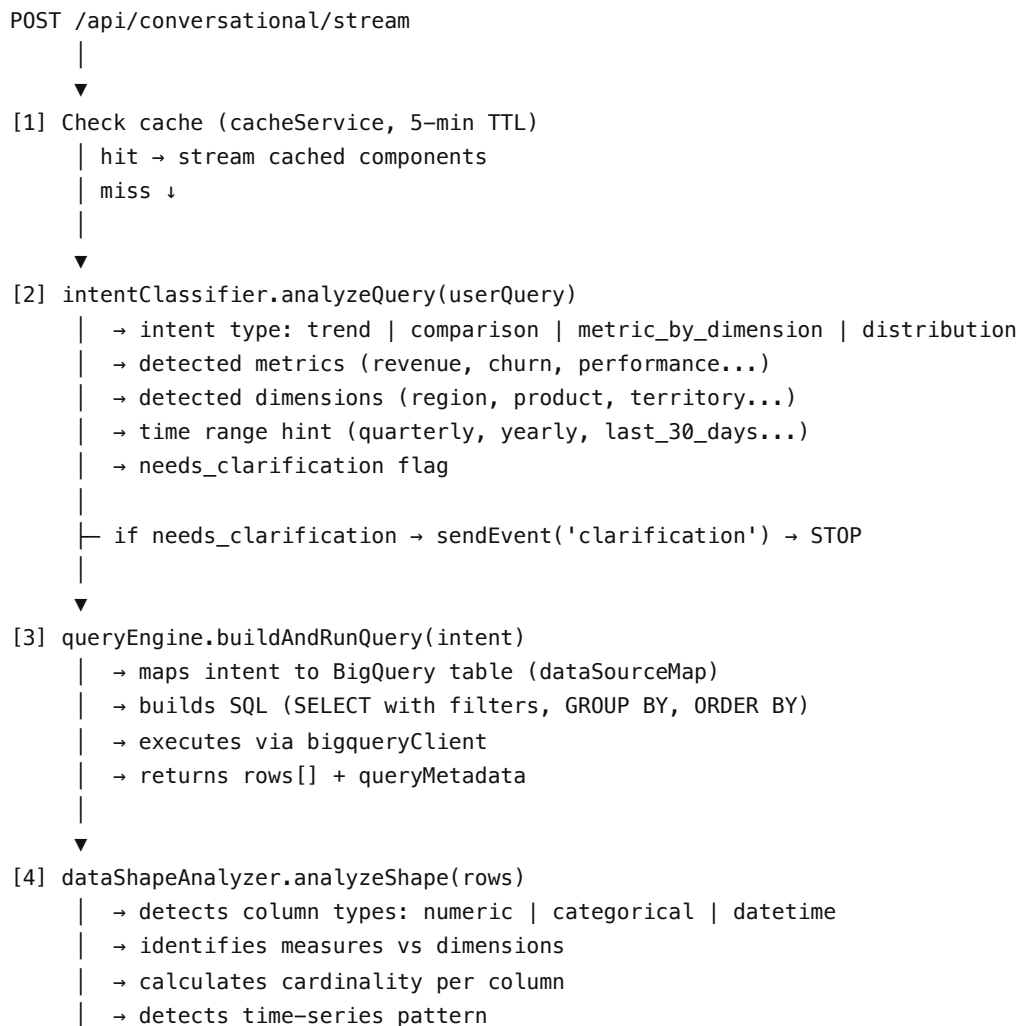
- [src/app/context/PersonaContext.tsx](#) — persona (`marketing_director` / `power_user`) drives quick actions + intent cards

4. Message Send Flow (Frontend)





5. Backend Pipeline — `runStreamingPipeline.ts`



```

    | → returns ShapeSignature
    |
    ▼
[5] llmHandler.generateReport(userQuery, shapeSignature, sampleRows[0..20])
    | → Gemma-4-31b-it call
    | → returns UITypeTree: { title, description, cards[] }
    | → each card: { renderType, props, children[] }
    | → retry on 429/500 (exponential backoff, max 3 attempts)
    |
    ▼
[6] fixColumnCasing(uiTree, actualColumns)
    | → BigQuery returns UPPERCASE columns
    | → LLM may lowercase them
    | → aligns all column refs in UI tree to actual BQ schema
    |
    ▼
[7] hydrateTree(uiTree, rows)
    | → attaches actual row data to each chart/table node
    | → aggregates time-series (dedup x-axis labels)
    | → deduplicates RankedList items
    |
    ▼
[8] Stream events to client
    | sendEvent('meta',      { title, description })
    | sendEvent('component', hydratedNode) ← one per card
    | sendEvent('followUp',  suggestedPrompts[])
    └─ cache result

```

6. SSE Event Protocol

All events sent as `text/event-stream` from Express to the browser fetch reader.

```

event: meta
data: { "title": "...", "description": "..." }

event: component
data: { "renderType": "LineChart", "props": {...}, "data": [...] }

event: followUp
data: { "prompts": ["...", "...", "..."] }

event: clarification
data: { "question": "...", "options": ["...", "..."] }

event: error
data: { "message": "..." }

```

Frontend parses each chunk:

```

const [eventLine, dataLine] = chunk.split('\n')
const eventType = eventLine.replace('event: ', '')

```

```
const payload = JSON.parse(dataLine.replace('data: ', ''))
```

7. LLM Integration — `llmHandler.ts`

Three LLM call types:

Function	When	Model Input	Output
<code>analyzeQuery()</code>	Every message	query + catalog context	intent classification + clarification flag
<code>generateReport()</code>	After BQ query	query + data shape + 20 sample rows	UITypeTree (card layout)
<code>classifyAndEditReport()</code>	Follow-up edits	existing report + user edit request	modified UITypeTree (no new BQ call)

Retry logic:

```
attempt 1 → if 429 or 500 → wait 1s → attempt 2 → wait 2s → attempt 3 → throw
```

8. BigQuery Data Sources — `dataSourceMap.ts`

Table	Domain	Primary Metrics
<code>fact_sug_monthly_rollup</code>	Sales	revenue, units, quota attainment
<code>v_monthly_territory_performance</code>	Territory	rep performance, pipeline
<code>v_daily_sales_detail</code>	Transactions	daily orders, ASP
<code>v_churn_analysis</code>	Retention	churn rate, cohorts
<code>v_product_performance</code>	Product	SKU revenue, margin
<code>v_customer_segments</code>	CRM	segment breakdown
<code>v_forecast_vs_actual</code>	Planning	forecast accuracy
<code>v_marketing_attribution</code>	Marketing	channel ROI, leads

Intent classifier maps user query → table via 50+ keyword synonyms.

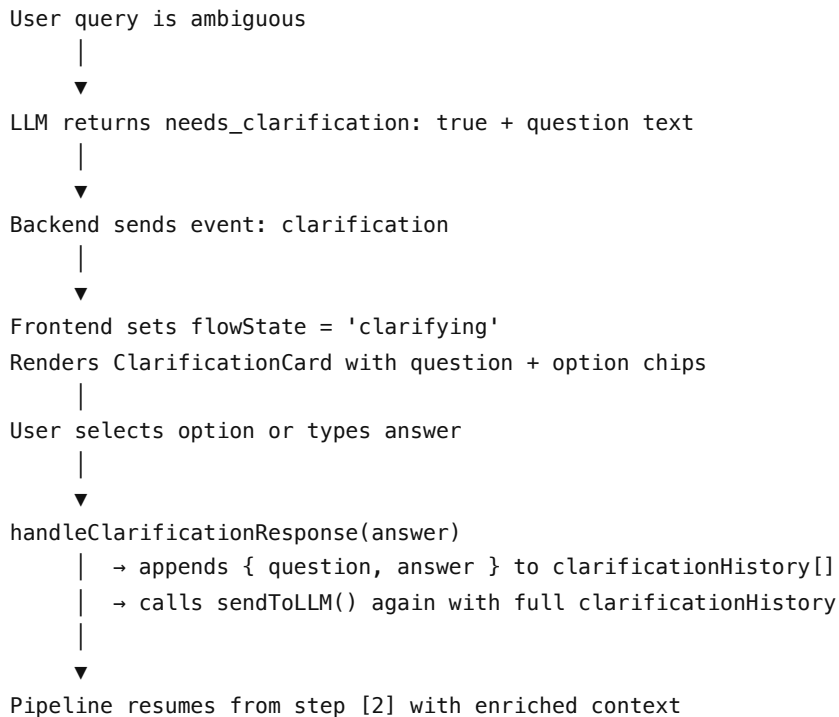
9. UITreeRenderer — Component Dispatch

```
UITreeRenderer receives { renderType, props, data }  
|  
├ "LineChart"      → Recharts LineChart  
├ "BarChart"       → Recharts BarChart  
├ "AreaChart"      → Recharts AreaChart  
└ "PieChart"       → Recharts PieChart
```

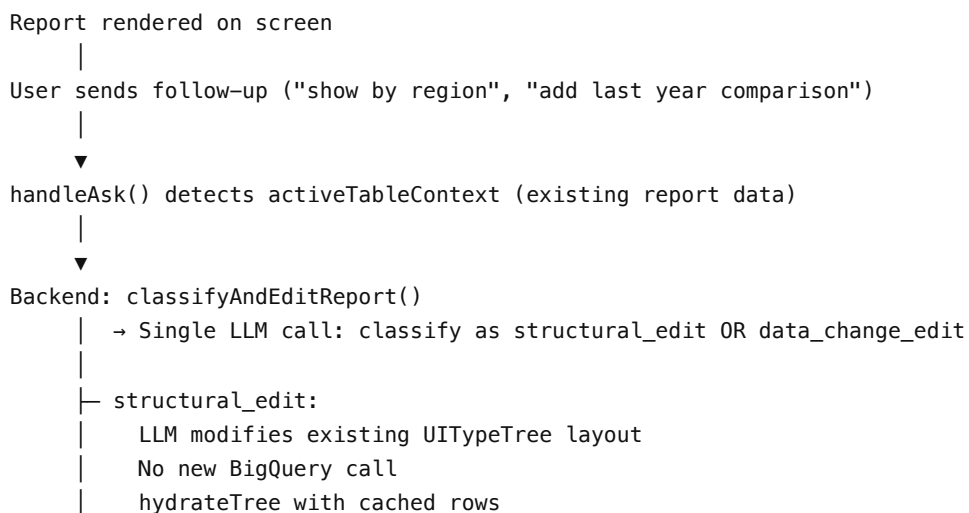
└ "Table"	→ GenerativeTable (sort/filter)
└ "KPICard"	→ Metric card with delta badge
└ "KPIGrid"	→ Grid of KPICards
└ "RankedList"	→ Sorted list with bar indicators

All chart types use design system tokens from MASTER.md (warm off-white surface, brand accent `#D4572A` , category colors).

10. Multi-Turn Clarification Flow



11. Follow-Up / Edit Flow



```
|
└─ data_change_edit:
    New BigQuery query with updated filters
    New LLM report generation
    Full pipeline re-runs
```

12. Caching — `cacheService.ts`

```
Key = stable hash of: userQuery + persona + conversationId
TTL = 5 minutes (default), 10 minutes (catalog)
```

```
On hit: skip steps [3]–[7], stream cached components
On miss: run pipeline, store result after step [8]
```

Catalog refreshes every 24 hours at server startup via `catalogRefresher.ts`. Catalog = markdown of all BigQuery domains, table schemas, report inventory — injected as LLM system context.

13. Conversation Persistence

```
Conversation
├─ id: uuid
├─ title: string (auto-generated from first query)
├─ status: 'draft' | 'active' | 'planned'
├─ messages: Message[]
│   └─ role: 'user' | 'assistant'
│   └─ content: string
│   └─ components: UINode[] ← hydrated chart/table data
│   └─ suggestedPrompts: string[]
└─ createdAt / updatedAt
```

Stored in component state; sidebar lists all conversations. Selecting conversation restores full message history including rendered components.

14. Error Handling

Layer	Error	Handling
LLM quota	429	Retry ×3 exponential backoff
LLM server error	500	Retry ×3 exponential backoff
LLM empty response	Empty cards	Fallback card generator runs
BQ query failure	SQL error	event: error to frontend
Column mismatch	LLM lowercase vs BQ uppercase	<code>fixColumnCasing()</code> normalizes
Network drop	SSE disconnect	Frontend shows error state, retry button

15. Persona-Based Customization

PersonaContext provides role-specific defaults:

Persona	Default Metrics	Quick Actions
marketing_director	channel ROI, leads, campaign performance	"Show pipeline", "Compare channels"
power_user	all available, raw data access	"Run custom query", "Export BQ SQL"

Injected into LLM system prompt so generated reports prioritize relevant metrics per role.

Key File Index

File	Purpose
src/app/pages/Conversational_new.tsx	Chat UI + all frontend state
src/app/components/UITreeRenderer.tsx	Renders all generative components
src/app/context/PersonaContext.tsx	Role-based context
backend/src/index.ts	Express server + route definitions
backend/src/pipeline/runStreamingPipeline.ts	Master orchestrator
backend/src/services/llmHandler.ts	Gemma LLM calls
backend/src/services/intentClassifier.ts	Query intent + clarification
backend/src/services/queryEngine.ts	BigQuery SQL builder + executor
backend/src/services/dataShapeAnalyzer.ts	Column type + cardinality analysis
backend/src/services/cacheService.ts	TTL in-memory cache
backend/src/services/catalogRefresher.ts	24h catalog refresh
backend/src/services/dataSourceMap.ts	8 BigQuery table definitions
backend/src/lib/bigqueryClient.ts	BQ connection + query execution
backend/src/types/index.ts	Shared TypeScript interfaces